

Si utilizas IA en 2026, debes comprender estos 17 conceptos

Guía para principiantes sobre los conceptos y la terminología de la IA.

[Hamza Boulahia](#)



En los últimos dos años, el debate sobre la IA ha cambiado por completo. Antes bastaba con saber qué eran los chatbot y los LLM.

¿Ahora?

La gente usa términos como RAG agentico, integraciones MCP, modelos de razonamiento, arneses de agentes e ingeniería de contexto, todo al mismo tiempo. Y todos en la sala asienten como si los entendieran.

Pero aquí está el problema de asentir con la cabeza...

Funciona bien hasta que eres tú quien tiene que tomar las decisiones.

Hasta que seas tú quien decida si comprar una herramienta, confiar en un resultado o explicarle a tu equipo por qué el asistente de IA acaba de hacer algo increíblemente extraño.

Es entonces cuando se hace evidente la brecha. Y ahora mismo, esa brecha se está ampliando día a día.

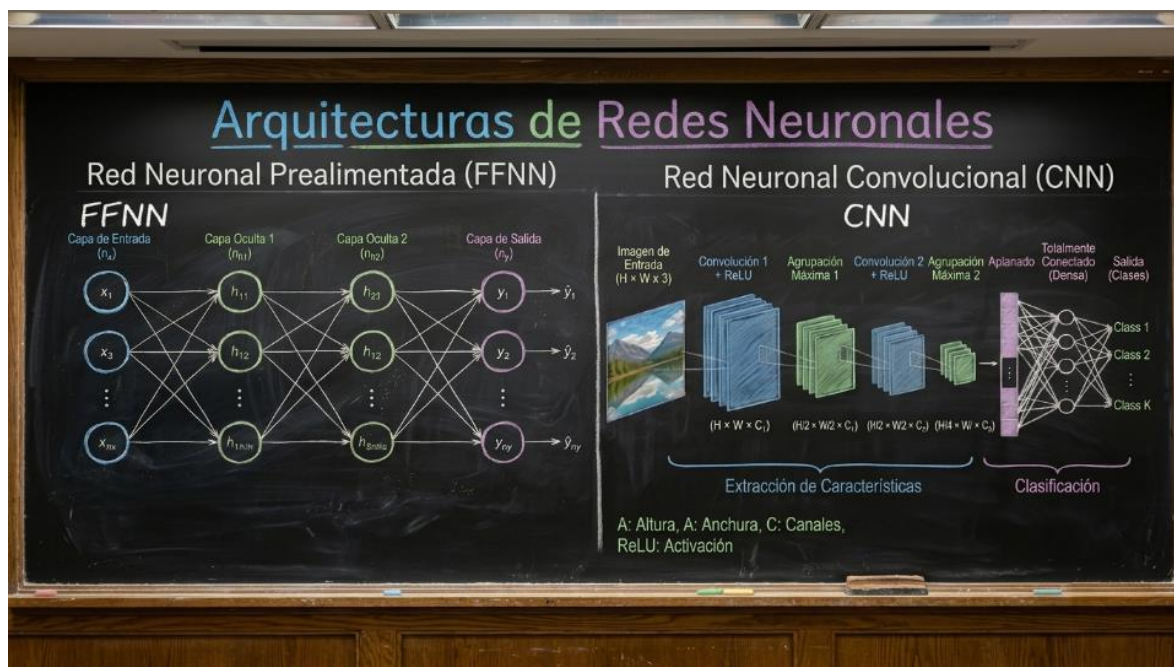
Pero la verdad es que la IA no es tan complicada.

Cuando empiezas a aprenderlo, puede resultar abrumador, pero comprender algunos conceptos esenciales puede darte un gran impulso en tu proceso de aprendizaje.

En esta publicación, explicaré 17 conceptos esenciales de IA de una manera que resulte comprensible para un principiante. Comenzaremos con la forma más primitiva de IA y avanzaremos gradualmente hacia conceptos más avanzados.

Esto no es un diccionario, ni una lista de palabras de moda para memorizar antes de una reunión.

1. Redes neuronales



Creo que la forma más sencilla de explicarlo es la siguiente: una **red neuronal** es, en realidad, una pila gigante de operaciones matemáticas básicas organizadas en capas.

Sin pensar.

Sin comprender.

Solo aritmética a gran escala.

Digamos que quieres que reconozca la foto de un gato:

1. Conviertes los píxeles en números y los introduces.
2. Los números fluyen a través de las capas de la red, multiplicándose y recombinándose una y otra vez.
3. Por el otro extremo sale un único número que indica el grado de confianza que tiene la red en que se trata de un gato.

Pero ¿dónde está la parte "inteligente"?

En una red neuronal entrenada para la clasificación de imágenes, las primeras capas detectan de forma natural los bordes, las capas intermedias las formas y las últimas los objetos completos. Lo fascinante es que esto no se programó manualmente; simplemente surgió de los cálculos matemáticos durante el entrenamiento del modelo.

Para entrenar una red neuronal:

1. Comienza con una red totalmente aleatoria y sin ninguna idea.
2. Muéstrale una foto y deja que adivine, va a ser una pésima elección.
3. Dile exactamente qué tan “lejos” estuvo de la respuesta correcta.
4. Ajusta ligeramente la configuración interna para que funcione mejor la próxima vez.

La configuración interna de cualquier red neuronal funciona como la perilla de una radio: se va girando y ajustando hasta encontrar la frecuencia correcta para el canal de radio deseado. Sin embargo, en las redes neuronales, ese ajuste se realiza automáticamente y de forma matemática precisa, mediante un método llamado **descenso de gradiente**, y las perillas del modelo se denominan **pesos**.



Bucle de entrenamiento de redes neuronales

Si se repite ese proceso millones de veces con diferentes datos de entrada, una máquina que empezó adivinando a ciegas acaba pareciendo un genio.

Lo realmente asombroso de las redes neuronales modernas, especialmente de los modelos de lenguaje natural (LLM), es que poseen **miles** de millones de pesos perfectamente ajustados para construir textos con sentido, palabra por palabra.

En cada iteración, el modelo debe **predecir** la siguiente palabra a partir de una colección de aproximadamente 200 000 posibles. ¡Y repite este proceso una y otra vez hasta generar la respuesta completa!

2. Tokens y tokenización



Los modelos de IA no leen texto como nosotros; no leen palabras ni letras. En cambio, leen **tokens**.

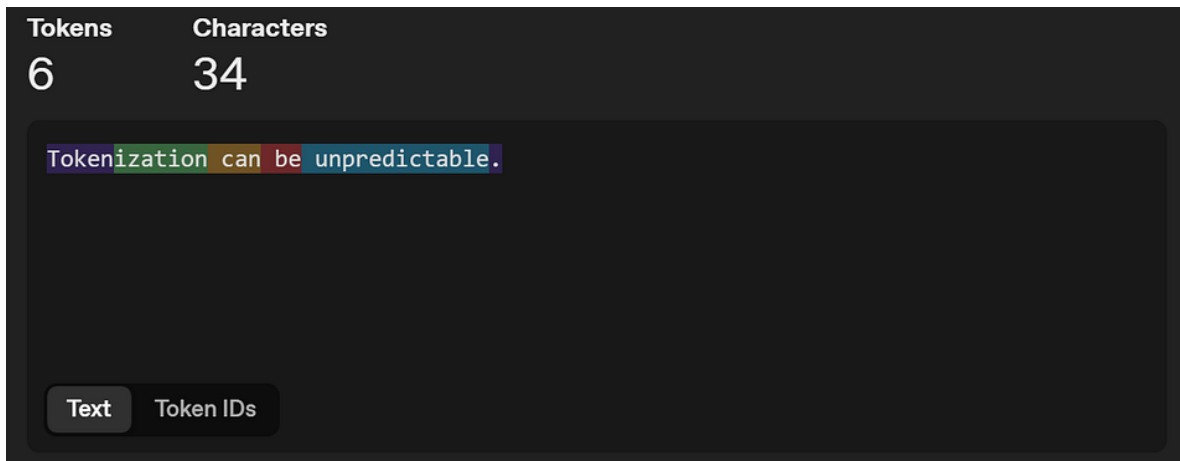
Y ese hecho explica gran parte del comportamiento de estos modelos.

Bien, ¿y ahora **qué es un token**?

Un token es simplemente un fragmento de texto, que puede variar entre un solo carácter y una palabra completa.

Tomemos como ejemplo la frase: *“La tokenización puede ser impredecible”*.

Un modelo no ve cinco palabras y un punto. Normalmente lo divide en seis partes distintas: *Token* , *ización* , *puede* , *ser* , *impredecible* y el punto.



Ejemplo de tokenización de GPT 5.x

¿Por qué debería importarte algo tan específico?

Porque todos los productos de IA que utilizas (por ejemplo, ChatGPT, Claude, Gemini, etc.) cuentan tokens en segundo plano. Y ese conteo determina prácticamente toda tu experiencia.

Cuanto más tokens incluyas en tu solicitud, más tendrá que trabajar el modelo. Cuanto más tokens genere, mayor será el coste de ejecución.

Los precios de la API se cotizan en tokens, y los "límites de contexto" también se miden en tokens.

Esta es precisamente la razón por la que las conversaciones largas acaban provocando que nos olvidemos de ellas.

Una vez que se agota el presupuesto de tokens del modelo, algo tiene que ceder. Y lo que cede suele ser el comienzo mismo de la conversación.

Esa es básicamente la razón por la que debería importarte comprender la tokenización desde un punto de vista práctico, pero la tokenización también es la raíz de varios problemas bien conocidos con los LLM.

Problema	Causa Raíz de Tokenización	Síntoma Típico
Fallos de Ortografía	Los modelos ven IDs de tokens, no caracteres individuales.	No puede contar letras; tiene dificultades con los anagramas.
Debilidad Matemática	Los números se dividen arbitrariamente basándose en la frecuencia.	Falla en sumas y multiplicaciones de varios dígitos.
Tokens de Glitch	Los artefactos en el vocabulario del tokenizador carecen de suficiente entrenamiento del modelo.	Alucinaciones extrañas o fallos desencadenados por palabras específicas.
Sensibilidad a Mayúsculas/Minúsculas y Errores Tipográficos	Cambios menores alteran completamente la fragmentación de tokens.	Un simple error tipográfico reduce drásticamente la calidad del razonamiento.
Sesgo de Idioma	Los tokenizadores están optimizados principalmente para el texto en inglés.	El texto que no está en inglés es más lento, más caro y menos preciso.

Problemas de los LLM derivados de la tokenización

3. Incrustaciones y búsqueda vectorial



Ya hemos establecido que los LLM se basan en tokens en lugar de palabras, pero también sabemos que las computadoras no entienden el significado, sino los números.

Las incrustaciones son el puente entre ambos.

Un incrustamiento es básicamente una larga lista de números que representa una palabra, una oración o un documento completo como un punto específico en el espacio.

El truco para dar significado a estos números reside en cómo se organiza ese espacio.

Así, cuando se entrena una IA, las ideas con significados similares acaban agrupadas, mientras que las que no guardan relación entre sí quedan muy separadas.

Por ejemplo, la palabra "*rey*" se sitúa justo al lado de "*reina*" y "*monarca*", y lejos de "*bicicleta*" o "*tormenta*". A diferencia de un tokenizador, que simplemente divide el texto según reglas, un **modelo de incrustación** aprende dónde colocar estos conceptos durante su entrenamiento.

Sé que suena un poco abstracto, pero espera a ver lo que realmente hace posible.

Búsqueda vectorial

Una vez que el significado se convierte en una ubicación física, encontrar información relacionada es simplemente cuestión de medir la distancia entre dos puntos.

Si tomas una pregunta, la conviertes en ese mismo tipo de lista de números y buscas los números más cercanos, obtienes lo que se llama **búsqueda vectorial**.

Esta es precisamente la razón por la que un bot de atención al cliente puede encontrar el artículo de ayuda adecuado incluso cuando formulas tu pregunta de forma totalmente diferente al texto original. Esto nos permitió pasar de la búsqueda **sintáctica** (es decir, basada en la sintaxis) a la búsqueda **semántica** (es decir, basada en el significado) .

Los modelos de incrustación representan el motor silencioso que impulsa la búsqueda semántica, los feeds de recomendaciones y la recuperación de datos.

Cuando la gente dice que una IA "entiende" tus archivos, lo que suele ocurrir internamente es mucho más sencillo:

- Tu texto se ha convertido en coordenadas.
- El sistema simplemente está analizando qué más hay cerca.

No se trata de una comprensión profunda. Es geometría básica y, francamente, funciona de maravilla.

4. Atención



Aquí hay una frase que causa problemas en los sistemas lingüísticos más antiguos y sencillos: *"El trofeo no cabía en la maleta porque era demasiado grande"*.

¿A qué se refiere "eso"? ¿Al trofeo o a la maleta?

Como humanos, podemos inferirlo instantáneamente. Pero un modelo tiene que calcularlo. El mecanismo que le permite hacerlo se llama **atención**.

El mecanismo de atención permite que el modelo analice cada palabra de una oración y determine la importancia de cada una para comprender la palabra específica que está procesando. A continuación, asigna ponderaciones a estas palabras en función de su relevancia.

En esa oración, la atención otorga un peso alto a "trofeo" y un peso bajo a "maleta".

¿Pero por qué? Pues porque los trofeos son las cosas que no caben cuando son demasiado grandes, no el recipiente.

Ahora, si cambiamos solo una palabra en nuestro ejemplo: *“El trofeo no cabía en la maleta porque era demasiado **pequeño**”*.

De repente, la balanza se inclina completamente hacia la "maleta".

Seguimos teniendo la misma estructura, pero la respuesta es opuesta. La atención lo consigue reevaluando las conexiones cada vez en función del contexto exacto.

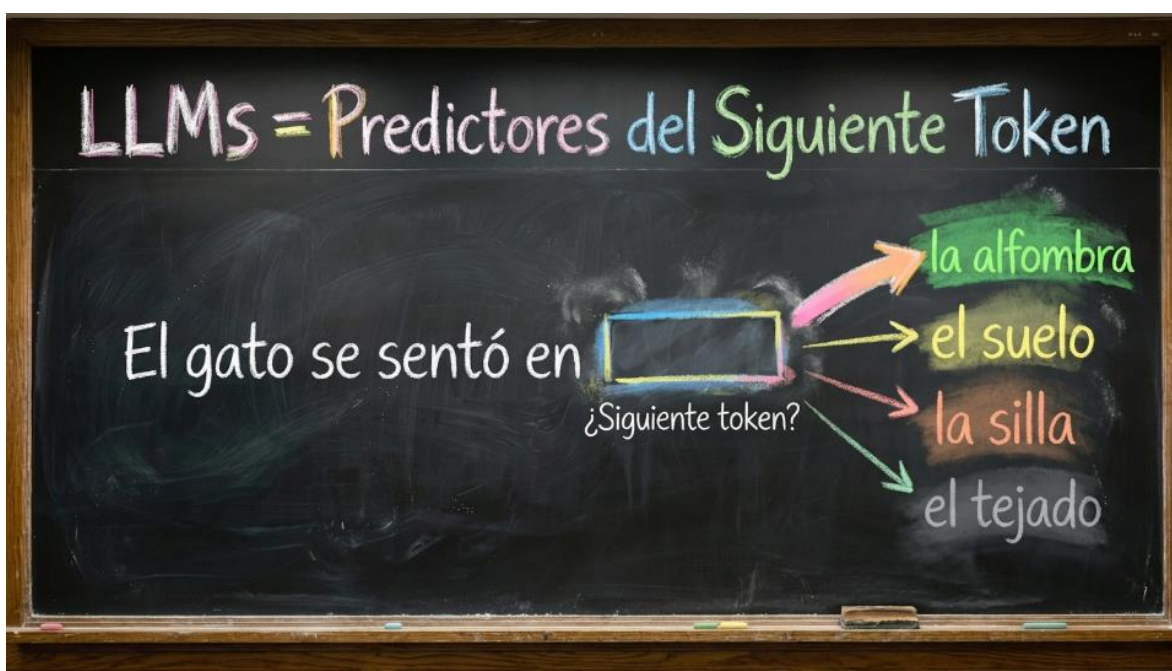
Este es precisamente el avance que hizo posible la IA moderna.

Antes de la atención, los modelos procesaban el texto principalmente en orden, palabra por palabra. Tenían serias dificultades para conectar ideas que estaban muy distantes en una oración o párrafo.

La atención permite que cada palabra mire directamente a todas las demás y evalúe la conexión en consecuencia.

También es de donde proviene la "T" de GPT. La arquitectura se llama **Transformer**, y la atención es la parte que realiza la transformación propiamente dicha.

5. LLM (Modelos de lenguaje a gran escala)



Los modelos de lenguaje a gran escala (LLM, por sus siglas en inglés) son redes neuronales entrenadas para realizar una sola tarea:

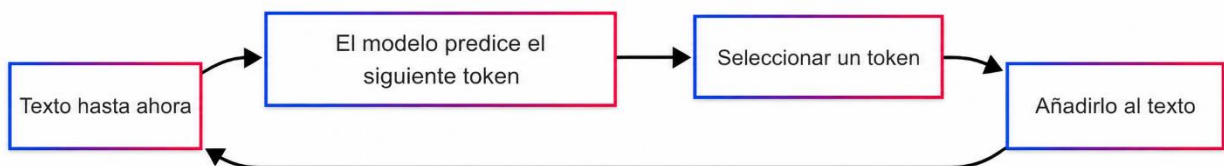
Predice el siguiente token.

Esa es toda la base.

Todas las conversaciones profundas, el código complejo, los ensayos, todo eso se basa en ese único truco, llevado a un nivel absurdo.

Así es como se ve eso en la práctica.

1. Le proporcionas un texto al modelo.
2. Analiza todo lo que has escrito hasta ahora y calcula una probabilidad para cada posible siguiente token.
3. Calcula la probabilidad matemática exacta de que el siguiente fragmento sea "el", "gato" o un punto, entre cientos de miles de opciones.
4. Selecciona uno y lo añade al texto.
5. Repite todo el proceso para el siguiente token. Luego el siguiente. Luego el siguiente...



Bucle de autocompletado de texto de LLM

Una respuesta que te parece instantánea, en realidad se está construyendo pieza a pieza, docenas o cientos de veces por segundo.

La parte “grande”, en los programas de LLM, se refiere a dos cosas:

- El tamaño de la red que realiza la predicción (a menudo cientos de miles de millones de parámetros)

- La enorme cantidad de datos de entrenamiento (normalmente una gran parte de todo Internet).

Y cuando entrenas un modelo tan grande para predecir el siguiente token en semejante cantidad de texto, **sucede algo extraño**.

El modelo comienza a aprender gramática, datos, patrones de razonamiento y estilos de escritura. No porque alguien haya programado esas reglas, sino porque predecir con precisión el texto humano a gran escala lo "obliga" a comprender cómo funciona el mundo.

Pero, en esencia, sigue siendo un modelo de autocompletado.

Simplemente, el autocompletado se tragó todo Internet.

6. Temperatura



En la sección anterior, hemos visto que los LLM predicen el siguiente token, basándose en la atención que calculan.

Internamente, asigna un valor de probabilidad a cada posible siguiente ficha antes de tomar su decisión final.

Aquí es precisamente donde entra en juego la **temperatura**.

Una **temperatura baja** (cercana a 0) significa que el modelo casi siempre elige el token siguiente más probable. El resultado es seguro, consistente y algo predecible. Si le pides que complete *"el gato se sentó en el..."*, obtendrás *"alfombra"* casi siempre.

Una **temperatura elevada** hace que el modelo ignore la opción obvia y se arriesgue con fichas menos probables. Con esa misma indicación, podrías obtener *"alféizar de la ventana, mesa"*.

Si lo pones demasiado alto, obtendrás *"el imperio en ruinas del martes"*. Es creativo, sin duda, pero completamente inútil si solo necesitas una respuesta directa.

Por eso, la gente suele referirse a la temperatura como **el parámetro de la creatividad**.

Aquí tienes una regla práctica sobre cómo ajustar la temperatura:

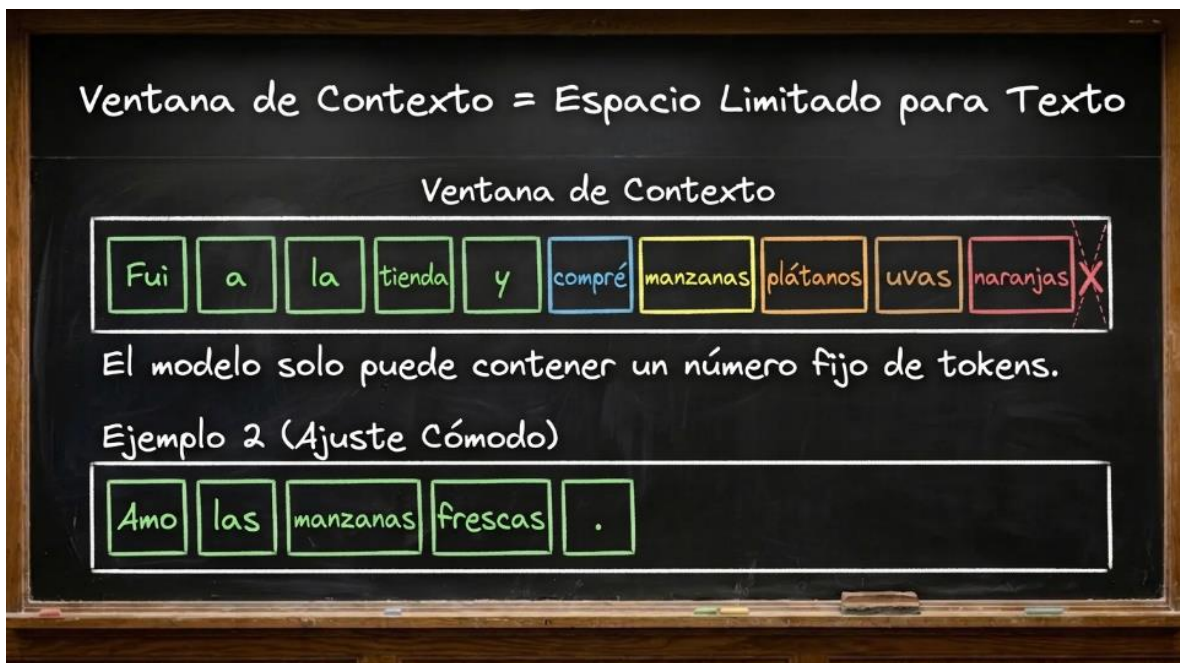
- Si estás resumiendo un documento, escribiendo código o extrayendo datos de un contrato, necesitas una temperatura baja. La precisión importa más que la

novedad, y definitivamente no quieres que el modelo improvise datos.

- Si estás generando ideas de marketing, escribiendo ficción o redactando textos publicitarios, querrás que sea más alto, para permitir ideas más creativas.

La mayoría de las aplicaciones para consumidores ocultan este parámetro por completo, fijándolo en un punto intermedio seguro en tu nombre. Pero en cuanto analizas una API o una herramienta para desarrolladores, ahí está (normalmente configurado con un valor entre 0 y 2). Y decide silenciosamente cuántos riesgos puede asumir el modelo con su siguiente acción.

7. Ventana de contexto



Si un LLM es un cuaderno, la ventana de contexto es su número de páginas.

Puedes escribir lo que quieras, pero en cuanto se llene, tendrás que borrar algo antiguo para dejar espacio para algo nuevo.

Eso es precisamente lo que es una **ventana de contexto**.

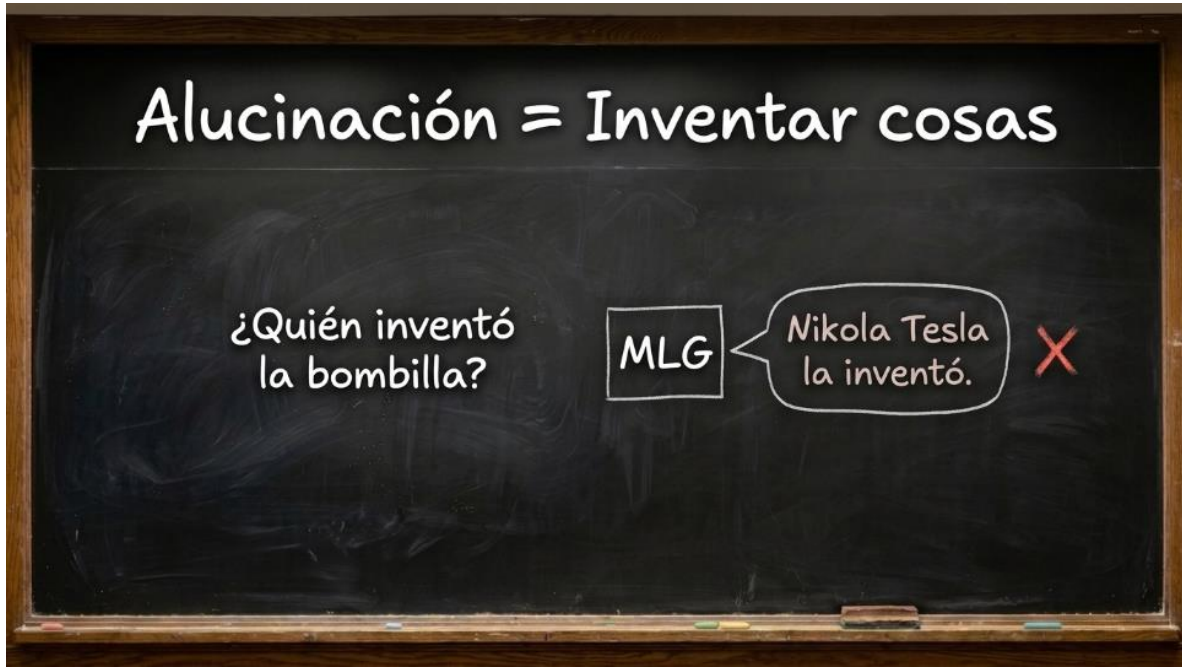
Es la cantidad máxima de texto (medida en tokens) que un modelo puede almacenar en su memoria activa en un momento dado.

La ventana de contexto puede contener tu mensaje actual, todo el historial de chat, cualquier documento extenso que hayas arrastrado y soltado en la ventana, y las respuestas anteriores del propio modelo.

Así, una ventana de contexto pequeña podría contener solo unas pocas páginas de texto antes de llenarse por completo. Pero la ventana de contexto de un modelo de última generación puede contener un libro de texto completo y aún así tener espacio de sobra para tus preguntas.

Si tu asistente de IA olvida repentinamente un detalle crucial que mencionaste hace veinte minutos, no se ha vuelto loco, simplemente revisa el indicador de la ventana de contexto, *puede* que esté llena.

8. Alucinación



Pregúntale a una IA sobre un libro que en realidad no existe.

No te dirá que no lo sabe. En cambio, te dará el título, el autor, el año de publicación y un resumen completo de la trama, con absoluta seguridad.

¡Todo completamente inventado!

Todos hemos experimentado esto, especialmente con los LLM de mayor edad.

Eso es **una alucinación**.

Una respuesta rotundamente errónea, presentada como una certeza absoluta.

Y aquí viene lo interesante... esto no es un fallo técnico.

Como hemos visto hasta ahora, los LLM no son bases de datos. No realizan búsquedas. Simplemente predicen el siguiente token más probable basándose en patrones.

Así que, cuando desconocen un dato, no se detienen. Simplemente siguen generando palabras que *suenan* correctas.

Lo que realmente hace peligrosas las alucinaciones es la forma en que se producen:

- Sin dudarlo.
- Sin cambio de tono.

Una respuesta errónea dicha con nerviosismo es fácil de detectar. Pero una respuesta errónea presentada como una cita pulida de Wikipedia es increíblemente fácil de creer.

El mejor consejo que he escuchado sobre este tema es el siguiente:

Considera las afirmaciones sobre IA como un consejo de un desconocido demasiado confiado... Es un buen punto de partida, pero siempre compruébalo dos veces.

9. Ajuste fino



Un **modelo de lenguaje base** es esencialmente un modelo que ha sido entrenado con toda la información de Internet.

Sabe un poco de todo, pero no es especialista en ningún tema ni tarea.

El ajuste fino es el proceso de tomar ese modelo general y someterlo a un programa de capacitación intensivo, específico y especializado.

Le proporcionamos un conjunto de datos más pequeño y cuidadosamente seleccionado de ejemplos específicos sobre la tarea que queremos que domine (por ejemplo, asistente conversacional).

Si necesitas un modelo que redacte informes médicos, analice contratos legales en un formato corporativo específico o imite el tono exacto de una marca, puedes perfeccionar un modelo base. Es un proceso mucho más económico que entrenar un modelo desde cero.

Esta es la trampa en la que la gente suele caer:

El ajuste fino no sirve para enseñar nuevos datos a una IA.

Es una trampa porque, técnicamente, se puede hacer, pero a estas alturas hay formas mucho mejores de lograrlo.

Si quieres que un modelo conozca el inventario actualizado de tu empresa o los documentos internos de RR. HH., no necesitas ajustarlo. Existe una forma mejor y más rápida de hacerlo (por ejemplo, **la Generación Aumentada de Recuperación (RAG)**), pero hablaremos de eso más adelante.

Ten en cuenta que el ajuste fino consiste en enseñar al modelo un nuevo **estilo, formato o tarea especializada** .

10. Barandillas y alineación (RLHF)



Un modelo de lenguaje básico, también conocido como **modelo base**, justo después del entrenamiento inicial, dirá cosas realmente incoherentes.

Eso se debe a que lo único que aprendió a hacer fue predecir el siguiente texto plausible, pero no aprendió a comportarse correctamente.

Las medidas de protección y la alineación son las capas que convierten un predictor de siguiente token sin procesar en algo con lo que realmente querrías interactuar.

La técnica principal que subyace a esto se llama **RLHF o Aprendizaje por Refuerzo a partir de la Retroalimentación Humana**.

La idea principal es, en realidad, bastante simple:

1. El modelo genera varias respuestas posibles a una pregunta.
2. Los humanos los califican: *este es servicial, aquel es grosero, este es peligroso*.
3. Esas valoraciones entrenan un " **modelo de recompensa** " independiente para predecir qué aprobarán los humanos.
4. El modelo original se ajusta para producir el tipo de respuestas que obtienen una alta puntuación en el modelo de recompensa.

Las barandillas de seguridad se colocan justo encima, como una segunda capa.

Se trata de **reglas** y **filtros** explícitos que bloquean ciertas categorías de resultados, independientemente de lo que el modelo subyacente pretenda generar.

- Piensa en la **alineación** como una forma de enseñar al modelo a emitir juicios sólidos a través de la retroalimentación.
- Piensa en **las barandillas** como muros físicos rígidos que no dependen en absoluto del criterio personal.

Los modelos alineados aún pueden ser persuadidos para que se comporten mal con los trucos de inducción adecuados, eso se llama **Jailbreaking**.

11. Ingeniería de prompts



Por defecto, los modelos generativos tienden a converger hacia el promedio estadístico de lo que han aprendido. Si se le pide a un modelo

de imagen básico entrenado con 1000 imágenes de gatos que genere un gato, este tiende a producir una versión combinada y "promedio" de esos gatos, lo que se conoce como el problema de la "**regresión a la media**".

Del mismo modo, sin indicaciones específicas ni ajustes aleatorios, los modelos de lenguaje grande (LLM) recurren por defecto a las palabras estadísticamente más probables, lo que produce un texto que resulta genérico, predecible y poco inspirado.

La diferencia entre ese borrador inútil y una respuesta realmente brillante no suele ser culpa del modelo.

Es la indicación (prompt).

La ingeniería precisa consiste en escribir instrucciones que realmente le indiquen al modelo lo que se desea, en lugar de simplemente hacer gestos vagos al respecto.

Una buena pregunta no es solo eso, una pregunta.

Incluye contexto crítico:

- ¿Para quién es esto?
- ¿Cuál es el objetivo final?
- Restricciones estrictas como la duración o el tono.
- Un ejemplo concreto de lo que significa "bueno".

Existe una técnica que merece la pena conocer por su nombre: **la técnica de indicaciones con pocos disparos**.

Aquí es donde le muestras al modelo un par de ejemplos exitosos antes de hacerle tu pregunta real.

Por ejemplo, ¿quieres que clasifique los tickets de soporte en categorías específicas? Primero, muéstrale dos o tres ejemplos etiquetados y luego pídele que etiquete uno nuevo.

Los sistemas de gestión del lenguaje modernos son excelentes para detectar patrones en las indicaciones del usuario y utilizarlos para mejorar su respuesta.

12. Mensajes del sistema



¿Recuerdas los GPT personalizados dentro de la aplicación ChatGPT?

Permitían a cualquiera definir instrucciones personalizadas, crear una personalidad muy específica, incorporar archivos de conocimiento adicionales y compartir el bot final con el público.

En su interior, todas y cada una de esas creaciones personalizadas utilizaban exactamente el mismo modelo base.

Simplemente estaban trabajando con **indicaciones del sistema** únicas, algunas herramientas personalizadas y un conjunto diferente de documentos de referencia.

Un **mensaje del sistema** es un conjunto oculto de instrucciones básicas que un desarrollador escribe antes de que usted escriba una sola palabra.

Normalmente no lo ves, pero influye en cómo se comporta el modelo durante toda la conversación.

Puede exigir una personalidad específica, imponer límites estrictos (por ejemplo, *nunca dar consejos médicos, siempre remitir a un profesional*) o enumerar las herramientas exactas que el modelo tiene permitido usar.

Tus mensajes de usuario habituales simplemente se añaden sobre esta base.

Esto tiene importancia práctica porque puede explicar muchos comportamientos aparentemente extraños de la IA:

- **Rechazos repentinos:** Una norma de seguridad oculta provocó un cierre forzoso.
- **Obsesiones persistentes:** Se trata de cumplir con el mandato de mantener el tema en el buen camino.

- **Cambios de personalidad:** Su papel se definió de manera diferente entre bastidores.
- **Uso oculto de herramientas:** Consiste en ejecutar una instrucción invisible para ejecutar código o navegar por la aplicación.

13. Ingeniería de contexto



La ingeniería rápida solía ser la clave del éxito.

Escribe una buena instrucción y obtendrás una buena respuesta.

Pero hoy, en la era de los agentes, eso ya no es suficiente.

Los sistemas con capacidad de respuesta automática que realizan trabajo en el mundo real ya no se limitan a reaccionar a una sola orden.

Ellos son:

- Cargando archivos externos.
- Llamando a herramientas especializadas.
- Recordando semanas de conversaciones pasadas.
- Enrutamiento de solicitudes a diferentes modelos según la tarea.

Entonces, para lograr que un agente recuerde todo eso y aún así funcione, necesitamos administrar adecuadamente su **ventana de contexto**, y eso es lo que llamamos **ingeniería de contexto**.

Mientras que la ingeniería de mensajes busca optimizar un único mensaje para el usuario, la ingeniería de contexto busca optimizar toda la estructura de la ventana de contexto.

Por eso ocurre en el código y en el flujo de datos que rodea al modelo.

Y si eres usuario de un agente de codificación (por ejemplo, Claude Code, Codex, OpenCode), necesitas comprender cómo funciona esto para reducir tu consumo de tokens.

La calidad de un producto de IA depende cada vez menos del modelo en sí que se encuentra en el centro, y mucho más de la eficacia con la que el sistema que lo rodea decide qué es lo que ese modelo realmente llega a ver.

Un modelo brillante, si se le proporciona el contexto equivocado, seguirá dando una respuesta completamente mediocre.

14. RAG (Generación aumentada por recuperación)



En la sección de ajuste fino dije que RAG es la mejor alternativa para enseñar nueva información a un LLM. Pero ¿cómo podemos hacerlo sin entrenar el modelo?

Un modelo de IA convencional solo conoce lo que aprendió durante el entrenamiento.

Eso significa que no sabe absolutamente nada sobre los documentos internos de su empresa, las noticias de la semana pasada o el PDF que acaba de subir.

¿Cómo responde entonces una herramienta como "NotebookLM" a preguntas sobre contenido que el modelo nunca ha visto realmente?

Esto se debe a que utiliza **la Generación Aumentada por Recuperación o RAG**.

A pesar de su nombre intimidante, se trata de una idea realmente sencilla.

Cuando subes un documento, el sistema no intenta comprimir todo el archivo en la ventana de contexto del modelo.

En cambio, divide el documento en pequeños fragmentos y los almacena dentro de una base de datos especializada llamada **Vector Store (o base de conocimiento)**, que es básicamente la base de datos que nos permite consultar el **espacio de incrustación**.

Cuando haces una pregunta, el sistema busca en esa base de datos los fragmentos más relevantes para lo que acabas de preguntar.

Extrae esas piezas específicas y se las entrega al modelo junto con tu pregunta: *aquí está el contexto relevante, ahora responde usando solo esto*.

Esto hace que el modelo dependa menos de su memoria y más del contexto proporcionado, lo cual es mucho más fiable para su uso en el mundo real.

Esa es toda la cadena de suministro.

1. Recuperar las piezas relevantes.
2. Introdúzcalos en el modelo.
3. Genera una respuesta basada en datos reales.

Esta es la columna vertebral oculta detrás de casi todos los productos de IA útiles creados recientemente.

A diferencia del ajuste fino, en RAG el modelo subyacente no ha aprendido nada nuevo.

Simplemente recibe la información precisa en el momento preciso.

Imagínelo como un examen a libro abierto en lugar de una prueba de memoria.

15. Agentes de IA



Un chatbot estándar responde a una pregunta e inmediatamente se detiene, esperando a que escribas la siguiente indicación.

Un agente no se detiene.

Establécele un objetivo de alto nivel y seguirá resolviendo el problema paso a paso, de forma totalmente autónoma, hasta que la tarea esté completa o se quede sin bucles para seguir intentándolo.

La arquitectura central que subyace a la mayoría de los agentes se llama **ReAct**, abreviatura de **Reasoning** plus **Acting** (razonamiento más actuación).

En lugar de un único paso directo de la entrada a la salida, funciona como un bucle continuo.

El agente analiza el estado actual de la tarea y razona sobre qué hacer a continuación.

Suele actuar contactando con una herramienta externa y llamándola.

Observa el resultado bruto de esa acción.

Luego, vuelve al razonamiento, armado con esta información completamente nueva, para decidir cuál es el siguiente paso lógico.

Este ciclo (es decir, **razonar, actuar, observar, repetir**) continúa hasta que decide que tiene todo lo necesario para darte una respuesta final.

Por ejemplo, un asistente de codificación con IA que depura código comienza leyendo el problema y recorriendo el código para detectar qué está fallando.

Una vez que tiene una corazonada, escribe una solución y la pone a prueba. Si algo falla, no se detiene, sino que lee los registros de errores, ajusta su enfoque y vuelve a intentarlo.

Cada nueva pista moldea su siguiente movimiento hasta que finalmente el insecto es aplastado.

16. MCP (Protocolo de Contexto de Modelo)



Hace unos años, si querías que un modelo de IA consultara tu calendario, tenías que crear una integración personalizada para ese modelo específico y esa aplicación de calendario específica.

¿Querías que también leyera el correo electrónico? Tenías que crear otra integración personalizada.

Imagínese **N** herramientas diferentes multiplicadas por **M** modelos diferentes, donde cada combinación requiere su propio conector; eso es una verdadera pesadilla.

MCP, el **Protocolo de Contexto de Modelo**, se introdujo precisamente para solucionar ese problema.

Se trata de una forma estandarizada para que los modelos de IA se conecten a herramientas y fuentes de datos externas, funcionando

exactamente igual que el protocolo USB-C estandarizó la forma en que los dispositivos se conectan a cargadores y periféricos.

Antes de la llegada del USB-C, cada teléfono parecía necesitar su propio cable propietario. Después, un único conector funcionó a la perfección en casi todos los dispositivos.

MCP hace lo mismo con la IA:

Una vez que se crea una herramienta o se expone una fuente de datos a través de MCP, cualquier modelo de IA compatible puede utilizarla al instante sin necesidad de una integración personalizada para cada combinación.

Por eso, este término aparece repentinamente en casi todas las presentaciones técnicas de productos que se escuchan entre 2025 y 2026.

Se trata de una infraestructura estandarizada que permite a un agente de IA acceder a tu calendario, tu almacenamiento de archivos, tu gestor de proyectos y una docena de otras herramientas sin necesidad de que un equipo de ingeniería construya y mantenga un conector independiente y frágil para cada una de ellas.

17. Arneses del agente



Si dejas que un agente de IA autónomo se ejecute completamente dentro de un script básico, eventualmente fallará, se quedará atascado en un bucle infinito o agotará tu presupuesto de API en cuestión de minutos.

Un modelo básico puede razonar, pero no tiene ni idea de cómo gestionar de forma segura su propio entorno de ejecución.

Un **arnés de agente**, también llamado **andamiaje de agente**, es la capa de infraestructura que envuelve al modelo para mantenerlo bajo control.

Si el agente es el cerebro, el arnés es el sistema nervioso y la cabina de mando.

La fórmula es sencilla: **Agente = Modelo + Arnés.**

El arnés es responsable de ejecutar el bucle de ejecución principal:

- Establece límites absolutos sobre la cantidad de pasos que un agente puede dar antes de verse obligado a detenerse.
- Gestiona la recuperación de errores, de modo que si una API de terceros agota el tiempo de espera, el sistema detecta el error y proporciona al modelo un mensaje claro para que vuelva a intentarlo, en lugar de permitir que toda la aplicación falle.
- También gestiona el estado de las indicaciones, eliminando el historial antiguo o resumiendo las acciones pasadas para que la memoria del agente se mantenga intacta y dentro de su presupuesto de tokens.

Aquí es donde se hace realidad la ingeniería a la hora de construir una IA de nivel de producción.

No basta con darle una instrucción a un modelo y esperar lo mejor. Hay que construir un contenedor robusto e instrumentado a su alrededor.